Embedding Model Training

How Sentence-BERT and OpenAI's embedding models actually learn: contrastive loss, triplet loss, in-batch negatives, and hard negative mining

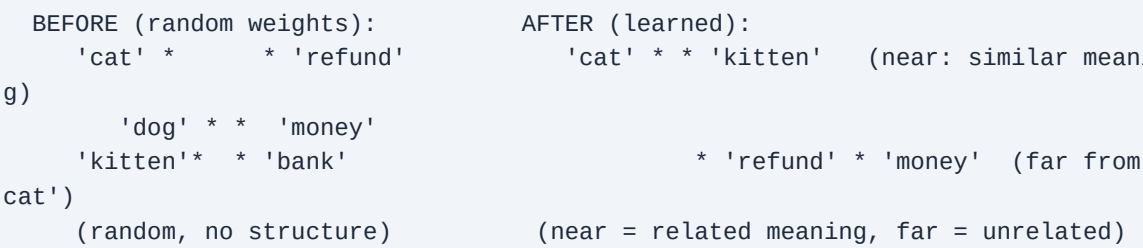
Concept Guide 9 of 9 | Read before: Embedding Model Training Zero-to-Hero lab | Companion to the Zero-to-Hero series

---

1. What 'Training an Embedding' Means

Earlier guides used FIXED embeddings (TF-IDF, hand-built concept maps) — no learning involved. Real embedding models (Sentence-BERT, OpenAI's embedding models) instead LEARN the text-to-vector mapping from data, via gradient descent, with a loss function designed for one specific goal.

DIAGRAM — Before vs. after training



MENTAL MODEL

Training an embedding model is slowly nudging a giant cloud of points in space so that MEANING becomes GEOMETRY — one gradient step at a time, exactly like the training loop from the PyTorch guide, just with a similarity-based loss instead of a classification loss.

PRACTICE THIS → **Embedding_Model_Training_Zero_to_Hero.ipynb Ch.1**

2. Contrastive Loss

Contrastive loss directly encodes the training goal: pull positive (similar) pairs together, push negative (dissimilar) pairs apart, up to a margin.

FORMULA — Contrastive loss

$$L = y*(1 - sim) + (1-y)*max(0, sim - margin)$$

y=1 for a positive pair, y=0 for a negative pair; sim is cosine similarity

WORKED EXAMPLE — Reading the two cases

POSITIVE pair (y=1): loss = 1 - sim. Loss is LOW when sim is HIGH (already close) -- good.

NEGATIVE pair (y=0): loss = max(0, sim - margin). Loss is ZERO once sim drops below the margin -- no more pushing needed once they're 'far enough' apart.

MENTAL MODEL

The margin means negative pairs don't need to be pushed to literally opposite vectors — just far enough apart that they're clearly distinguishable. Wasting effort over-separating already-distant pairs adds nothing useful.

PRACTICE THIS → **Embedding_Model_Training_Zero_to_Hero.ipynb Ch.3**

3. Triplet Loss

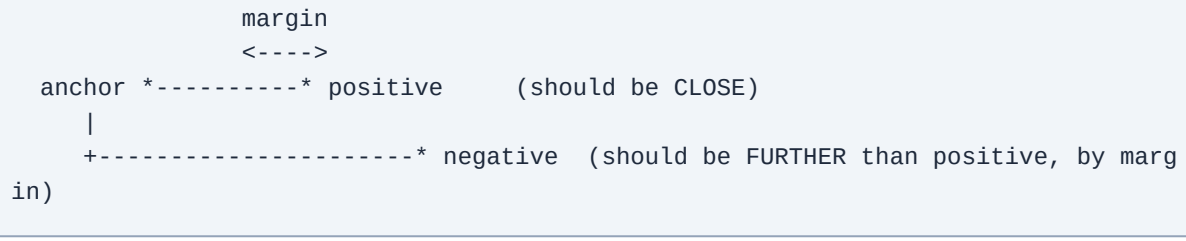
Triplet loss is contrastive loss's more common cousin in practice. Instead of labeled pairs, it uses triplets: an anchor, a positive (similar to the anchor), and a negative (dissimilar).

FORMULA — Triplet loss

$$L = \max(0,\ d(anchor, positive) - d(anchor, negative) + margin)$$

a RELATIVE constraint: negative must be further from anchor than positive, by at least margin

DIAGRAM — The triplet relationship



COMMON MISUNDERSTANDINGS

- If a negative is chosen randomly from a huge, diverse dataset, it's often already trivially far from the anchor — giving ZERO loss and zero learning signal. This is exactly why hard negative mining (Section 5) matters.

PRACTICE THIS → **Embedding_Model_Training_Zero_to_Hero.ipynb Ch.4**

4. In-Batch Negatives: Training at Scale

Manually labeling explicit negative pairs for every training example doesn't scale. In-batch negatives is the trick real models use: within a batch of N positive pairs, treat every OTHER pair's partner as a free negative.

DIAGRAM — Free negatives from batch structure

```
batch of positive pairs: (q1,a1), (q2,a2), (q3,a3)

for q1: a1 is the positive; a2 and a3 automatically become negatives
for q2: a2 is the positive; a1 and a3 automatically become negatives
for q3: a3 is the positive; a1 and a2 automatically become negatives
-> N positives and N*(N-1) negatives, WITHOUT labeling a single extra example
```

FORMULA — In-batch contrastive loss

```
similarity_matrix = queries @ docs.T / temperature
loss = cross_entropy(similarity_matrix, labels=[0,1,2,...,N-1])
```

the CORRECT match for query i is doc i -- the diagonal of the similarity matrix

MENTAL MODEL

This is exactly why embedding-model training uses LARGE batch sizes: bigger batches mean more free negatives per training step, a stronger and more efficient learning signal per gradient update.

PRACTICE THIS → **Embedding_Model_Training_Zero_to_Hero.ipynb Ch.5**

5. Hard Negative Mining

Random or in-batch negatives are often 'easy' — already far apart, giving a weak learning signal. Hard negative mining deliberately finds negatives that are CONFUSINGLY similar to the anchor but still technically wrong, forcing the model to learn finer distinctions.

DIAGRAM — Easy vs. hard negatives

```
anchor: 'how do I reset my password'

EASY negative:  'what is your refund policy'   (already very different -- weak
signal)
HARD negative:  'how do I reset my email'       (superficially similar wording,
                                                  wrong answer -- forces real l
earning)
```

MENTAL MODEL

Production pipelines mine hard negatives PERIODICALLY during training, using the model's CURRENT embeddings — as the model improves, what counts as 'confusingly similar' shifts, so the hard-negative set is refreshed rather than fixed once.

PRACTICE THIS → **Embedding_Model_Training_Zero_to_Hero.ipynb Ch.9**

6. Evaluating a Trained Embedding Model

After training, evaluate the same way as any search system: retrieval accuracy — for each query, does its true matching document rank highest by cosine similarity?

FORMULA — Top-1 retrieval accuracy

```
accuracy = fraction of queries where argmax(similarity to all docs)
                        == the true match
```

MENTAL MODEL

Always compare a trained model's accuracy against an UNTRAINED (random-weight) baseline — a raw number like '80% accuracy' is hard to interpret alone, but '80% vs. a 20% random baseline' clearly isolates how much the training itself contributed.

PRACTICE THIS → **Embedding_Model_Training_Zero_to_Hero.ipynb Ch.8**